OOP IN JAVA · MINI PROJECT → RESEARCH PAPER → PRODUCTION-GRADE SYSTEM

RaktSetu

Expiry-Aware Blood Redistribution Network

Final Implementation Roadmap · Version 2.1 (solo)

The one-line pitch
e-Raktkosh answers *"Where is blood available?"* RaktSetu answers *"Which units will expire unused, where should they go now, and how sure are we, given that stock reports are often out of date?"*

Author	Scope
Pritika Khandelwal	Solo project: design, implementation, research, report, presentation and demonstration

Contents

1. What changed from V1
2. The six deliverables at a glance
3. Phase plan and timeline
4. Architecture
5. Algorithms and research method
6. Phase-by-phase tasks
7. Deliverable 1: Project Report
8. Deliverable 2: UML Design Documents
9. Deliverable 3: Java Source Code
10. Deliverable 4: Executable Project
11. Deliverable 5: PowerPoint Presentation
12. Deliverable 6: Final Demonstration
13. Research paper (bonus deliverable)
14. Quality, risks and working rules
15. Evaluation mapping and viva preparation
16. References

1. What changed from V1

The first roadmap was a two-week course plan. This version keeps that plan as **Phase 1** (a safe, gradeable checkpoint) and extends the same codebase into a research contribution and a production-quality system. The six course deliverables stay the same; each one gets a stronger final version. This is a **one-person project**: the phases run one after another, and Section 3.1 says what to cut first if time runs short.

Area	V1 plan	Final plan	Why
Problem framing	Redistribute near-expiry units	Redistribute near-expiry units **when stock reports are stale or wrong**	A 2025 paper already combines substitution, transshipment and RL [7]. Stale data is the gap we found, and it is India's real problem [1].
Legality	"Recommends only"	Inter-blood-centre transfer is legal since the 3 April 2017 amendment to the Drugs and Cosmetics Rules [12]	Stronger viva answer; still human-approved.
Time	LocalDate.now()	An injected **Clock**; demo and simulation choose "today"	Sample data never goes stale on demo day; runs are reproducible.
Risk check	Per-unit rule, no usage data	FEFO-ordered cumulative rule using a recorded usage history	The V1 rule had no input data and miscounted.
Compatibility	Red cells and plasma only	All four components, incl. whole blood (ABO-identical) and platelets (ABO-identical preferred)	Clinically correct, fully tested (8 × 8 × 4 cases).
Strategies	Nearest-First, Expiry-First	8 policies, from "no transfer" to a stochastic optimiser and a perfect-information bound	Enough baselines for an IEEE-grade evaluation.
Audit log	Append-only text file	Event-sourced, **hash-chained** log; state is rebuilt from events	Tamper-evident; any past day can be replayed.
Simulation	30 days, 2 runs	India-calibrated digital twin, 365 days, 30 replications, 8 policies × 5 compliance levels	Statistically defensible results.
Interface	Console only	Console (course) + REST/FHIR server + web dashboard with a map	Shows system design depth.
Executable	JAR + run.bat	Fat JAR, Windows app-image with bundled Java (jpackage), Docker Compose	Runs on a lab PC with no Java installed.

2. The six deliverables at a glance

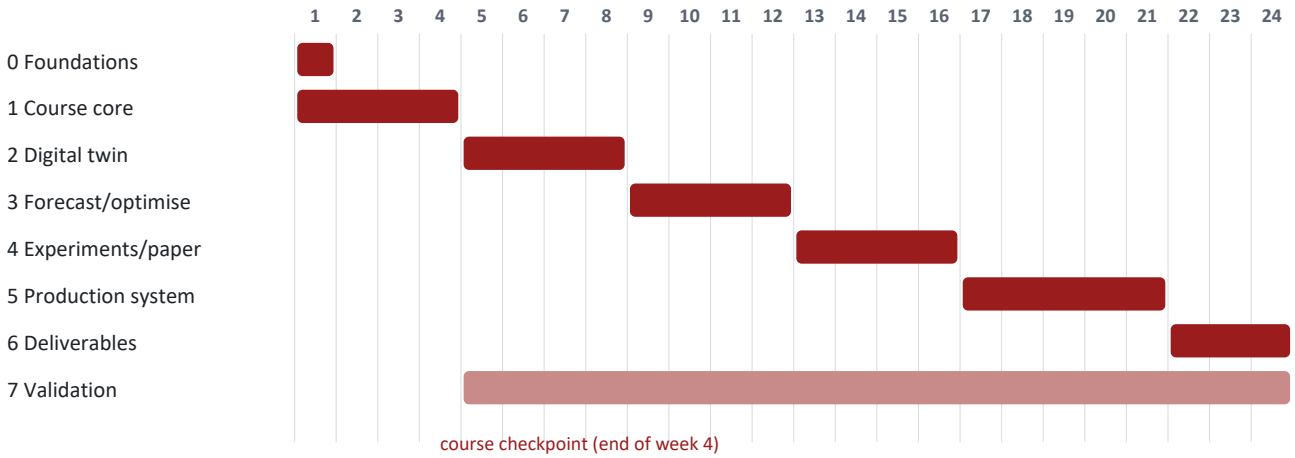
#	Deliverable	Final form	Course version	Final version
1	Project Report (PDF)	55–70 page report in LaTeX (or Word), IEEE citation style, 12 chapters + appendices	Week 4	Week 23
2	UML Design Documents	11 PlantUML diagrams kept in the repo, bundled as one PDF	Week 4 (v1)	Week 23
3	Java Source Code	Maven multi-module repo, Java 21, ~80% test coverage on core modules, CI green	v0.4 tag, Week 4	v1.0.0 tag, Week 21
4	Executable Project	raktsetu.jar, Windows app-image, run scripts, Docker Compose, sample data, README	JAR, Week 4	Week 21
5	PowerPoint Presentation	18 main slides + 6 backup slides	Week 4	Week 23

#	Deliverable	Final form	Course version	Final version
6	Final Demonstration	10-minute scripted demo, fully offline, fixed seed, recorded backup video	Week 4	Rehearsed Week 24
+	Research paper	6–8 page IEEE conference paper (IEEEtran) + arXiv preprint + open benchmark	—	Submit Week 16

If the course deadline is close
Submit the Phase 1 versions of all six deliverables at the end of Week 4, or compress Phase 1 to fit your deadline (the must-have row in Section 3.1 is enough to do well). Everything after Phase 1 upgrades the same repository and documents; nothing is thrown away.

3. Phase plan and timeline

Phase	Weeks	Goal	Exit gate
0 · Foundations	1	Repo, CI, domain model, design doc, data schemas, UML v1	Build green on GitHub (done)
1 · Course core (V1)	1–4	Working console app with all OOP concepts, tests, JAR	All 16 menu options work; 60+ tests pass; course-ready
2 · Digital twin	5–8	Event log, India-calibrated simulator, reporting-staleness model, baselines	Same seed gives byte-identical output; baselines run
3 · Forecast and optimise	9–12	Quantile forecaster with conformal calibration, stochastic optimiser, staleness-aware beliefs	Optimiser beats Expiry-First on a pilot run
4 · Experiments and paper	13–16	Full experiment grid, statistics, figures, paper written and submitted	All RQ results with 95% CIs; paper submitted
5 · Production system (V3)	17–21	Spring Boot server, Postgres event store, REST API, dashboard, load tests	p99 emergency search < 300 ms at target load
6 · Deliverables	22–24	Final report, UML, executable, PPT, demo rehearsal	Two clean end-to-end rehearsals on the lab PC
7 · Validation (V4)	5 → beyond	Partner blood centre, ethics approval, retrospective test on real data	Anonymised dataset received (stretch)



Week numbers are relative, assuming about 15–20 hours a week. Phases run in sequence because there is one person; only Phase 7 (emails and ethics paperwork) runs in the background. Write the report chapter for each phase as you finish it, so Phase 6 is assembly and polishing, not writing from scratch.

3.1 What to cut first if time runs short

Priority	Keep	Cut or simplify
Must have	Phase 1 in full: the course marks depend on it	Nothing
Should have	Phases 2–4: simulator, policies P0–P6, experiments, paper	Drop the PPO baseline (P7); use one network size instead of two
Nice to have	Phase 5: REST API and a simple dashboard	Drop FHIR, Keycloak and OpenTelemetry; keep the event store, the concurrency test and one load test
Stretch	Phase 7 real-data validation	Keep as future work in the report and paper

4. Architecture

4.1 Key decisions

Decision	Choice	Reason (say this in the viva)
Overall shape	**Modular monolith** (Maven modules with strict boundaries), not microservices	India-scale load fits one service (Section 4.4). Splitting too early adds failure modes with no benefit.
Language	Java 21 everywhere the course grades; Python only for the optional RL baseline and plots	The course requires Java source. OR-Tools has native Java bindings, so the optimiser stays in Java.
State	**Event sourcing**: each unit is a sequence of events; current stock is a projection	The audit log becomes the source of truth. Any past day can be rebuilt; simulations are replayable.
Audit integrity	SHA-256 hash chain over events	Editing any past entry breaks every later hash; the demo shows this.
Time	Injected Clock (SystemClock / SimulationClock)	Deterministic tests, stable demo data, reproducible experiments.
Same code for sim and live	Simulator drives the real engine through the same interfaces	Paper results come from the shipped code, not a separate prototype.
Unit identity	ISBT 128-style donation identification numbers	Global standard for labelling blood units [16].
Interoperability	FHIR R5 *BiologicallyDerivedProduct* read API (V3)	Hospitals' systems can consume it [16].
Maps	Travel-time matrix precomputed once with OSRM on OpenStreetMap; served by *DistanceProvider*	Real road times, zero network calls on demo day.
Privacy	No donor personal data stored at all	Keeps the system simple under India's DPDP Act 2023.

4.2 Module layout

```
raktsetu/  (Maven multi-module, Java 21)
|-- raktsetu-domain       model, interfaces, exceptions, compatibility, unit events
|                         (pure Java, zero dependencies)
|-- raktsetu-engine       allocation, risk, forecasting, freshness beliefs,
|                         strategies, optimiser (OR-Tools)
|-- raktsetu-persistence  CSV repositories (V1), JDBC event store (V3),
|                         hash-chained audit log
|-- raktsetu-sim          digital twin, scenarios, experiment runner, metrics
|-- raktsetu-cli          console application (course deliverable, 16 options)
|-- raktsetu-server       Spring Boot 3: REST + FHIR API, auth, reservations (V3)
|-- web/                  dashboard: React/Next.js + Leaflet map (V3)
|-- experiments/          configs, Python notebooks for figures, optional PPO
|-- data/                 sample data, calibration scripts, travel-time matrix
`-- docs/                 design doc, ADRs, uml/*.puml, report/, paper/, slides/
```

4.3 Runtime view (V3)

```
  Web dashboard (map, approvals, freshness, simulator)      Console app (V1)
                 |                                                  |
                 +---------- REST / FHIR  (OIDC, roles) ----------+
                 |
  raktsetu-server  ->  raktsetu-engine  ->  raktsetu-domain
    inventory, allocation, reservations, transfers, freshness,
    planner (forecast + optimiser), audit (hash chain)
                 |                                  |
  PostgreSQL + PostGIS                   OpenTelemetry -> Prometheus / Grafana
  (event store + projections)            (traces, metrics, SLO dashboards)
```

4.4 Capacity estimate

157.14 lakh units a year [1] is about **43,000 new units a day**, or roughly 0.5 writes per second on average. With ~5–7 events per unit lifetime that is under 5 events per second nationally, and the live stock (a few weeks of red cells plus days of platelets) is on the order of 1–2 million units across 5,679 facilities [11]. A single PostgreSQL instance handles this comfortably; the design scales by partitioning events by state if ever needed. Putting this estimate in the report shows engineering judgement.

5. Algorithms and research method

5.1 Clinical rules (single source of truth)

Component	Shelf life / storage (confirm against NACO/NBTC standards [15])	Compatibility rule implemented
Whole blood	35 days (CPDA-1), 2–6 °C	ABO- and Rh-identical only (contains both cells and plasma)
Red cells (SAGM)	42 days, 2–6 °C	Donor cells must lack antigens the recipient has antibodies to: O− universal donor, AB+ universal recipient
Platelets	5 days, 20–24 °C with continuous agitation	ABO-identical preferred; other groups allowed with a warning; Rh− preferred for Rh− recipients
Plasma (FFP)	1 year at −30 °C or colder; use within 6 h of thawing	Reversed: AB is the universal plasma donor, O plasma only to O recipients

Rules live in *BloodGroup* (*canGiveRedCellsTo*, *canGivePlasmaTo*); each component class delegates to the right rule. Expiry is defined once: *expiry = collectedOn + shelfLife*, and a unit is expired when *today ≥ expiry*. Checking compatibility returns a result with a reason; only issuing or transferring an unsafe unit throws

IncompatibleBloodException.

5.2 Emergency allocation (Phase 1)

1. Rank facilities by travel time from the requesting hospital; skip facilities whose data is marked unverified until the verified ones are exhausted.
2. At each facility, try the exact group first, then compatible substitutes ordered to conserve scarce groups (O− last for non-O− patients).
3. Issue **first-expiry-first-out (FEFO)** from a *PriorityQueue* ordered by expiry date.
4. Reserve units with a time-limited reservation before confirming, so two requests can never take the same unit (V3: optimistic locking + idempotency keys).
5. If total stock is insufficient, throw *InsufficientStockException* listing how many compatible units exist and where.

5.3 Expiry-risk check (Phase 1)

For each facility, component and group, sort units by expiry. With average daily usage u (from the recorded usage history, last 14 days), unit number k (1-based) is **at risk** if $k > u \times daysToExpiry(k)$: the centre cannot use k units before that unit expires. This gives an exact at-risk count, not a per-unit guess.

5.4 Demand forecasting (Phase 3)

- Target: daily demand per hospital × component × group; horizon 1–7 days (platelets) and 1–14 days (red cells) [18].
- Model: negative-binomial regression with day-of-week, holiday and recent-usage features (implemented in Java), with seasonal naive and exponential smoothing as baselines.
- Uncertainty: split **conformal calibration** of the predicted quantiles, giving distribution-free coverage guarantees [13].
- Metrics: MAE, pinball loss, coverage of 80%/95% intervals, and, for RQ4, the downstream wastage each forecaster produces.

5.5 Stale-data belief (the research contribution)

Each centre i last reported its stock Δ_i days ago. The true stock today is modelled as the reported stock minus the issues and discards that happened since, which are unknown but forecastable. The engine keeps a belief (a sampled distribution) over true stock whose spread grows with Δ_i. Planning uses samples from this belief instead of trusting the report. In the simulator, each centre reports on a given day with probability π_i (its **compliance**), so we can measure how the value of this idea changes as compliance falls.

5.6 Transfer optimiser (Phase 3)

Every simulated day the planner solves a two-stage stochastic program by sample-average approximation over scenarios s that jointly sample (a) true current stock from the belief and (b) future demand from the calibrated forecast.

Element	Definition
Sets	Facilities I, components C, blood groups G, remaining-life classes r, scenarios s in S (probability p_s)
First stage (today)	Integer transfers x_{ijcgr} of units with remaining life r from i to j; allowed only if travel time $\tau_{ij} < r$ and the unit is not reserved

Element	Definition
Second stage (next H days)	Per scenario: FEFO issues $y_{jcg'g,s}$ of donor group g' to recipient group g where compatible; shortages and outdates follow
Objective	min $\Sigma_s p_s$ [w_{out}·outdates + w_{short}·shortages + w_{sub}·substitutions + w_{O-}·O− used on non-O− patients] + w_{tr}·Σd_{ij}·x
Constraints	Stock balance with ageing, transfer ≤ believed stock, vehicle/day capacity, no transfer that creates a new at-risk surplus at the destination
Solver	OR-Tools MPSolver with SCIP (Java bindings); rolling horizon H = 3 days for platelets, 7 for red cells; |S| = 50–200
Explanation	Every recommended transfer carries its expected units saved, km, and the binding reason (e.g. "source surplus 9, destination shortfall 6")

5.7 Policies compared

#	Policy	Purpose
P0	No transfers, FEFO issuing	Status quo
P1	Nearest-First (V1)	Simple rule
P2	Expiry-First (V1)	Simple rule
P3	Reactive: transfer only on stock-out	Current practice
P4	Deterministic optimiser on point forecasts	Standard predict-then-optimise
P5	Stochastic optimiser, trusts reports (ablation)	Isolates the value of the belief model
P6	**Stochastic optimiser + stale-data belief (proposed)**	Our method
P7	PPO reinforcement learning (stretch, Python)	Compare with [7]
P*	Optimiser with perfect information	Upper bound on what is achievable

5.8 Research questions and experiment design

RQ	Question	Measured by
RQ1	How much wastage does proactive transfer prevent, per component?	Outdate rate by component, P0 vs P1–P6
RQ2	Does modelling stale data help, and how does its value change as reporting compliance falls from 100% to 20%?	P6 − P5 gap across compliance levels
RQ3	What is the trade-off between wastage, shortage, transport and rural/urban fairness?	Pareto front over cost weights; fill-rate gap rural vs urban
RQ4	Does a more accurate forecast give better decisions?	Forecast pinball loss vs downstream wastage

- **Network:** one calibrated region (proposed: Pune division, Maharashtra, the highest-discard state [1]): 1–2 hubs, ~15 blood centres, ~30 hospitals, plus a larger synthetic network for scale.
- **Calibration:** state collection volumes (data.gov.in [14]), discard reasons and shares (Rajya Sabha answer [1]), component discard rates from Indian studies [9][10], facility density and travel times [11].

- **Runs:** 365 days after a 30-day warm-up; 30 replications; **common random numbers** (every policy sees identical demand and supply).
- **Factors:** 9 policies × 5 compliance levels × 2 network sizes (+ cost-weight sweep for RQ3).
- **Statistics:** means with 95% confidence intervals; paired Wilcoxon signed-rank tests with Holm correction; effect sizes.
- **Honesty:** results are from simulation; state it, and list threats to validity (calibration choices, demand patterns not taken from Indian hospitals, no transport failures).

6. Phase-by-phase tasks

Phase 0 · Foundations (Week 1) · done

- GitHub repository, Maven multi-module build with the Maven wrapper, CI on Java 21 and 25, JaCoCo coverage.
- Domain model: blood groups and compatibility, components, facilities, FEFO inventory, exceptions; 42 tests.
- Design doc, architecture decision records, CSV formats, UML v1 (use case, domain classes).

Done when: CI is green on GitHub. **Completed 25 September 2026.**

Phase 1 · Course core, V1 (Weeks 1–4)

mponentType, abstract BloodComponent + 4 subclasses, abstract Facility + 3 subclasses, ityQueue per component×group), BloodRequest, TransferOrder, Reservation,	
iryTracked, Transfusable, RedistributionStrategy, Exportable, DistanceProvider, e, DemandForecaster, Clock	
ion + InsufficientStock, IncompatibleBlood, UnitExpired, FacilityNotFound, InvalidData, rvationConflict	
tends Identifiable>, CsvRepository<T> (parser: Function<String[],T>), DataLoader, rovider, AuditLogger (hash-chained), ReportWriter	
RequestService, ExpiryRiskAnalyzer, FreshnessTracker, RedistributionEngine, piryFirst	
on, fixed seed, same demand for both runs, daily collections at the hub, t	
(below), input validation, never crashes	
roperty tests: all 8×8×4 compatibility cases, FEFO order, expiry boundary, risk count, save/reload, determinism	All (cross-testing)

Console menu (16 options)	
1 View centre stock (urgent ≤ 2 days highlighted)	9 Approve or reject transfers
2 View network-wide stock	10 View audit log + verify hash chain
3 Add blood units	11 Run the 30-day simulation
4 Raise an emergency request	12 View the wastage summary
5 Check compatibility (with reason)	13 Export reports (CSV)

Console menu (16 options)	
6 Run the expiry-risk check	14 Data-freshness report per centre
7 Generate a redistribution plan (choose strategy)	15 Set simulation date (“today”)
8 Compare strategies side by side	16 Save and exit

Done when: every menu option works on sample data; invalid input never crashes; 60+ tests pass; restarting reloads the same state; the fat JAR runs with one command. **This is the course-safe checkpoint.**

Phase 2 · Digital twin (Weeks 5–8)

- Replace mutable stock with **sealed event records** (UnitCollected, UnitTested, UnitStored, UnitReserved, UnitIssued, UnitTransferred, UnitDiscarded) and projections; hash-chain every event.
- Discrete-time simulator in *raktsetu-sim*: daily collection, demand arrival, issuing, ageing, discards, transfers; all randomness from one seeded source per stream.
- **Reporting process:** the engine sees only what each centre reported (with compliance π_i and lag), while the simulator tracks the hidden true state.
- Calibration scripts (*data/*): state volumes, discard shares, component mix; travel-time matrix precomputed with OSRM from an OpenStreetMap extract.
- Experiment runner: YAML config → grid of runs → tidy CSV of metrics per run and day.
- Implement policies P0–P3.

Done when: the same config and seed produce byte-identical output; a 365-day run of the pilot network finishes in under a minute; P0–P3 results look plausible.

Phase 3 · Forecasting and optimisation (Weeks 9–12)

- Forecasters: seasonal naive, exponential smoothing, negative-binomial regression; conformal calibration; backtesting harness.
- Belief model: sample true stock from last report + forecast of unseen issues/discards over Δ_i days.
- Optimiser: SAA model in OR-Tools, policies P4–P6 and P*; per-transfer explanations; solver time limit with a greedy fallback.
- Optional (first thing to cut): PPO baseline P7 in Python reading the same simulator through a small file/socket interface.

Done when: on the pilot grid, P6 beats P2 on outdates without a worse shortage rate; the optimiser returns a plan within 10 s per simulated day.

Phase 4 · Experiments and paper (Weeks 13–16)

- Run the full grid (Section 5.8); store raw CSVs and the exact commit hash with every result.
- Figures in Python/matplotlib: outdate rate by policy and component, P6−P5 gap vs compliance, Pareto fronts, forecast-vs-decision plot.
- Write the paper in IEEEtran (Section 13); ask your guide or a faculty member to review it; post to arXiv; release the benchmark on GitHub with a DOI (Zenodo).

Done when: every claim in the paper points to a figure or table generated by a script from raw results.

Phase 5 · Production system, V3 (Weeks 17–21)

- *raktsetu-server*: Spring Boot 3, PostgreSQL + PostGIS event store (Flyway migrations), REST API documented with OpenAPI, FHIR BiologicallyDerivedProduct read endpoint.
- Security: OIDC login (Keycloak in Docker), roles CENTRE_STAFF, HOSPITAL_STAFF, NETWORK_ADMIN; every write is audited.
- Concurrency: reservations with expiry, optimistic locking, idempotency keys; a test fires 100 parallel requests for the last 10 units.
- Dashboard: map of centres coloured by risk and freshness, stock tables, transfer approval queue, simulator panel with a compliance slider.
- Observability: OpenTelemetry traces, Prometheus metrics, Grafana dashboard with SLOs; load test with k6.
- Integration tests with Testcontainers; Docker Compose brings up the whole stack with one command.

Done when: the concurrency test never double-issues a unit; the k6 test meets p99 < 300 ms for emergency search; *docker compose up* works on a clean machine.

Phase 6 · Deliverables (Weeks 22–24)

Build the six deliverables exactly as specified in Sections 7–12. Week 24 is rehearsal and buffer only.

Phase 7 · Validation, V4 (background from Week 5)

- Week 5: email 3–5 blood centres (start with teaching hospitals) asking for anonymised daily stock, issue and discard records.
- Obtain institutional ethics committee approval before touching any real data; sign a data-use agreement.
- If data arrives: retrospective replay (would RaktSetu's plans have reduced last year's outdates?). This moves the paper from conference to journal level.

7. Deliverable 1: Project Report (PDF)

Written in LaTeX (Overleaf, shared) or Word with a shared template; IEEE numbered citations; 55–70 pages plus appendices. Figures and tables come from the repository so they always match the code.

Chapter	Contents	Written in	Pages
Front matter	Title page, certificate, acknowledgements, abstract (250 words), contents, list of figures/tables	Phase 6	5
1 Introduction	Problem, headline figures [1], motivation, objectives, contributions, report outline	Phase 1	4
2 Background	Blood components, shelf lives, compatibility incl. plasma reversal, Indian regulation (2017 transfer amendment), e-Raktkosh	Phase 1	5
3 Literature review	Forecasting [4][5], platelet inventory [6][8], transshipment and RL [7], inaccurate inventory records [19], logistics [3]; comparison table; the gap	Phase 2	6
4 Requirements	Actors, use cases, functional and non-functional requirements (SLOs, privacy, offline operation)	Phase 1	4
5 System design	Architecture, module layout, event sourcing, ADRs, data model, all UML diagrams	Phases 1, 5	8
6 OOP design	Where each OOP concept is used, with code excerpts (Section 9.3 table)	Phase 1	5

Chapter	Contents	Written in	Pages
7 Algorithms	Allocation, risk rule, forecasting, belief model, optimiser formulation, complexity	Phase 3	7
8 Implementation	Module by module, key classes, interesting problems solved (concurrency, determinism)	Phases 1–5	6
9 Testing and QA	Test strategy, the full test table, coverage, property tests, load-test results	Phases 1, 5	4
10 Experimental evaluation	Setup, calibration, RQ1–RQ4 results with figures and confidence intervals	Phase 4	8
11 Discussion	Limitations (and why they exist), threats to validity, ethics, deployment considerations	Phase 4	3
12 Conclusion and future work	Summary, V4 validation, transport integration, e-Raktkosh data source	Phase 6	2
References	20–30 IEEE-style references, primary sources only	Ongoing	2
Appendices	A User manual · B CSV and API schemas · C Test logs · D Full UML · E Experiment configs · F Glossary	Phase 6	10+

Checklist: every figure numbered and cited in text; every number traceable to a source or a result file; screenshots taken during development (keep a *docs/screenshots* folder from Week 1); spell-check; ask a friend or your guide to read the introduction and results chapters.

8. Deliverable 2: UML Design Documents

All diagrams are PlantUML text files in *docs/uml/*, reviewed in pull requests like code and rendered to PNG/SVG by CI. v1 in Week 1, updated at the end of Phases 1 and 3, and regenerated in Week 23 to match the final code.

#	Diagram	What it shows
1	Use-case	Centre staff, hospital staff, network administrator, and the Simulator as a secondary actor
2	Package diagram	The six Maven modules and their allowed dependencies (no cycles)
3	Class diagram: domain	Components, facilities, inventory, requests; inheritance, interfaces, association, aggregation, composition with multiplicities and access modifiers
4	Class diagram: engine	Strategy pattern (9 policies behind RedistributionStrategy), forecasters, belief model, optimiser
5	Class diagram: persistence and sim	Repository<T>, event store, audit chain; simulator, scenarios, metrics
6	Sequence: emergency request	Search by travel time → exact match → substitute → reserve → issue → audit
7	Sequence: plan and approve	Risk check → belief sampling → optimiser → admin approval → re-validation → transfer events
8	State machine: blood unit	Collected → Tested → Available → Reserved → Issued / In transit → Available / Expired → Discarded
9	Activity: daily planning cycle	Reports in → freshness update → forecast → optimise → recommendations
10	Component and deployment (V3)	Server, database, dashboard, identity provider, observability; Docker Compose layout
11	ER / event schema	Tables for events, projections, facilities, reservations

Patterns to label on the diagrams: Strategy (policies), Repository, Factory (component creation from CSV), Observer (event projections), Template Method (abstract BloodComponent), Builder (requests).

9. Deliverable 3: Java Source Code

9.1 Standards

- Java 21: **records** for events and value objects, **sealed interfaces** for the event hierarchy, pattern-matching *switch* in projections.
- Google Java Style enforced by Checkstyle; SpotBugs clean; Javadoc on every public type.
- A branch and pull request for every change, even solo: CI must be green, and you review your own diff against the checklist in the template (an automated code review is a useful second pair of eyes).
- Coverage ≥ 80% on domain and engine (JaCoCo gate); no business logic in the console or controllers.

9.2 Test strategy

Level	Tool	Examples
Unit	JUnit 5, AssertJ	FEFO order; expiry boundary (platelets on day 5 and day 6); risk count; each exception thrown in a real situation
Property-based	jqwik	All 8×8×4 compatibility cases match a reference table; issuing never returns an expired unit; stock is never negative
Golden file	JUnit	Simulation with seed 42 produces exactly the stored CSV (determinism)
Persistence	JUnit	Save → restart → identical state; a corrupted CSV row raises InvalidDataException and other rows still load; tampered audit entry is detected
Integration (V3)	Testcontainers	API against a real PostgreSQL; FHIR endpoint returns valid resources
Concurrency (V3)	JUnit + executors	100 parallel requests for the last 10 units: exactly 10 succeed
Load (V3)	k6	Emergency search p99 < 300 ms at 50 requests/s

9.3 OOP concepts coverage

Requirement	Where it appears
Classes and objects	Facilities, units, requests, transfers, reservations, services (60+ types in total)
Constructors	Overloaded (unit with or without explicit expiry); chaining with this() and super(); records' compact constructors validate input
Inheritance	4 components extend BloodComponent; 3 facility types extend Facility; 8 exceptions extend RaktSetuException
Polymorphism	getShelfLifeDays() and canTransfuseTo() per component (plasma reversal); 9 strategies behind one interface; forecasters behind DemandForecaster
Abstraction	Abstract BloodComponent and Facility; engine depends only on interfaces
Interfaces	Transfusable, ExpiryTracked, RedistributionStrategy, Exportable, DistanceProvider, StockDataSource, DemandForecaster, Clock
Encapsulation	Inventory exposes no internal queues; immutable records for events
Exception handling	Custom checked hierarchy, handled at the menu/controller boundary with clear messages
File handling	CSV load/save, hash-chained audit log, report export, experiment output

Requirement	Where it appears
Collections	PriorityQueue (FEFO), EnumMap<BloodGroup, …>, HashMap, ArrayList, TreeMap (events by time)
Generics	Repository<T extends Identifiable>, CsvRepository<T>, Result<T>
Packages / modules	Six Maven modules with enforced boundaries
Association / aggregation / composition	TransferOrder links two facilities; BloodNetwork aggregates facilities; each Facility owns its Inventory
Modern Java	Sealed interfaces, records, pattern matching, streams, java.time, virtual threads in the experiment runner

10. Deliverable 4: Executable Project

Artifact	How it is built	How to run
raktsetu.jar (fat JAR)	maven-shade-plugin in raktsetu-cli	*java -jar raktsetu.jar --data data/sample --today 2026-10-01 --seed 42*
run.bat / run.sh	Checked into the repo	Double-click on Windows; *./run.sh* elsewhere
Windows app-image	**jpackage** with a bundled Java runtime	Unzip and run RaktSetu.exe; **no Java installation needed** on the lab PC
Docker Compose (V3)	Dockerfiles for server and dashboard; Postgres and Keycloak images	*docker compose up*, then open the dashboard in a browser
Sample data	Generator with a fixed seed: 1 region, ~200 units per centre, travel-time matrix	Regenerated relative to “today”, so it never expires before the demo
Release	GitHub Release v1.0.0 with checksums and changelog	README: requirements, quick start, troubleshooting, screenshots

Done when: a friend who has never seen it can run the app on a clean Windows PC in under 5 minutes using only the README.

11. Deliverable 5: PowerPoint Presentation

lakh units discarded, 46% from expiry [1]
latelets' 5 days
d its two gaps: no decisions, stale data
comparison table)
user
and key design decisions
s map
ained audit log
eservations
ecasting with conformal intervals

ser (one diagram, no heavy maths)	
ign	
)	
e data vs compliance (RQ2), trade-offs (RQ3)	
rage, load test, CI	
ork (V4 validation)	
-off	
esults, test table, UML sequence diagrams, capacity estimate, references	—

12 minutes of slides + 10 minutes of demo. One idea per slide, large figures, numbers from the final result files only.

12. Deliverable 6: Final Demonstration

Time	Step	What the examiner sees
0:00	Start the executable (app-image), data loads	Network of centres; stock with urgent units highlighted
1:00	O− red-cell emergency request	Served from the nearest compatible centre, FEFO, reservation, travel time shown
2:30	Plasma request	Reversed rule: AB plasma offered to an O recipient, explained
3:30	Over-large request	InsufficientStockException handled; message lists where stock exists
4:30	Freshness report	Two centres flagged as stale; engine treats their stock as uncertain
5:30	Risk check → plan (Expiry-First vs optimiser) → approve	Side-by-side plans with “units saved / km”; approval writes events
7:00	Audit log: verify, then tamper with one line and verify again	Hash chain detects the edit
8:00	Simulation / experiment results; compliance slider (dashboard, if V3 is ready)	Wastage curves for each policy; the stale-data gap
9:30	Wrap-up	One sentence on impact and limitations

Pre-flight checklist

- Run the full script twice on the actual lab machine the day before; fully offline; fixed seed; “today” set explicitly.
- Keep a screen recording of a perfect run and all screenshots on a USB drive as a backup.
- Font size large in the console; window layouts arranged in advance; notifications off.
- Practise answering questions while driving the demo yourself; there is no teammate to take over the keyboard.

13. Research paper (bonus deliverable)

Working title: *Transshipment of Perishable Blood Components under Stale Inventory Reports: A Stochastic Optimisation Approach and an India-Calibrated Benchmark.*

Section	Contents	Length
Abstract	Problem, method, key result, benchmark release	200 words
I Introduction	Indian wastage data, stale reporting, contributions (formulation, belief-aware optimiser, open benchmark)	1 page
II Related work	Forecasting, platelet inventory (MDP/ADP/RL), transshipment, inaccurate inventory records; the gap	0.75 page
III Problem formulation	Section 5.6 in full mathematical form	1 page
IV Method	Conformal forecasts, belief model, SAA optimiser, complexity	1 page
V Experimental setup	Calibration, network, policies, factors, statistics	0.75 page
VI Results	RQ1–RQ4 with figures and tables	1.5 pages
VII Discussion and limitations	Threats to validity, deployment, ethics	0.5 page
VIII Conclusion	Summary and future work	0.25 page

- **Target venues:** simulation-only results → IEEE CASE, IEEE EMBC, IEEE BHI or IEEE ICHI (conference). With real partner data → IEEE Journal of Biomedical and Health Informatics or IEEE Transactions on Automation Science and Engineering.
- **Before submitting:** complete a systematic literature search (IEEE Xplore, Scopus, PubMed) to confirm the stale-data gap; check the venue's page limit and deadline; ask your guide whether they should be a co-author, and check your institution's policy for course-project publications.
- **Reproducibility:** code, configs and results on GitHub with a Zenodo DOI; one command regenerates every figure.

14. Quality, risks and working rules

14.1 Working rules

- Weekly rhythm: Monday plan (GitHub Projects board), Friday run-through of what works, nothing “done” unless merged with green CI.
- Show progress to your guide every two weeks; outside feedback replaces the reviews a team would give.
- Feature freeze one week before submission; anything unfinished is cut, not squeezed in.
- Screenshots and result files are captured as you go, never recreated at the end.

14.2 Risk register

Risk	Likelihood	Mitigation
Scope too large for one person	High	Phase gates; Phase 1 is complete and gradeable on its own; the cut list in Section 3.1 decides what goes
Optimiser too slow	Medium	Solver time limit, fewer scenarios, per-component decomposition, greedy fallback

Risk	Likelihood	Mitigation
Results show no benefit of the belief model	Medium	Still publishable as a negative/characterisation result; report where it helps (low compliance)
No real data from a partner	High	Paper framed as a simulation study with an open benchmark; V4 stays future work
Novelty already published	Medium	Systematic literature search in Phase 2; adjust the contribution (benchmark, India calibration) if needed
Integration breaks late	Medium	Trunk-based development, CI on every push, integration tests from Phase 2
Demo fails on the day	Low	Offline app-image, rehearsals on the lab PC, recorded backup
Single point of failure (illness, exams)	Medium	Everything pushed to GitHub daily; buffer at each phase boundary; the cut list in Section 3.1

15. Evaluation mapping and viva preparation

Component (marks)	What earns the marks
Problem Definition and Innovation (10)	Parliamentary data from 2026, the stale-data gap, a novel belief-aware optimiser, an open benchmark
Java Implementation and OOP Concepts (15)	Every concept used naturally, modern Java, clean module boundaries, design patterns, tests
Final Working Application (15)	Stable menu, no crashes, persistence, tamper-evident audit, reproducible simulation, bundled executable
Presentation and Viva Voce (10)	Confident explanation of every module and design decision, honest limitations with design answers

Likely question	Suggested answer
Is this real data?	Simulated, but calibrated to published Indian figures; the engine only depends on interfaces, so a real data source is one new class. We have asked a blood centre for anonymised data.
Can blood legally be moved between centres?	Yes, since the 3 April 2017 amendment to the Drugs and Cosmetics Rules [12]. RaktSetu recommends; authorised staff approve; every step is audited.
What is new compared to existing research?	Earlier work assumes stock reports are correct. We model stale reports explicitly and measure how much that matters as compliance falls.
Why not microservices?	The national load is a few events per second (Section 4.4). A modular monolith is simpler and just as scalable here; modules can be split later.
Why a hash chain instead of a blockchain?	We need tamper-evidence, not distributed consensus. A hash chain gives that with no extra infrastructure.
Where is polymorphism?	Each component overrides shelf life and compatibility (plasma is reversed); nine strategies share one interface.
Composition vs aggregation?	A Facility creates and owns its Inventory (composition); BloodNetwork holds facilities that exist independently (aggregation).
Why FEFO with a PriorityQueue?	O(log n) insert and removal of the soonest-expiring unit; sorting a list on every issue would be O(n log n).
How do you know the results are not luck?	30 replications with common random numbers, 95% confidence intervals and paired significance tests.

16. References

[1] Factly (14 Aug 2026). Blood Collection Rose 24% in India, While Discards More Than Doubled. Based on e-Raktkosh data in reply to Rajya Sabha Unstarred Question No. 1882, answered 4 Aug 2026. factly.in

[2] Ministry of Health and Family Welfare. e-RaktKosh: Centralized Blood Bank Management System. eraktkosh.mohfw.gov.in

[3] Nisingizwe M.P. et al. (2022). Effect of unmanned aerial vehicle (drone) delivery on blood product delivery time and wastage in Rwanda. The Lancet Global Health, 2022.

[4] Guan L. et al. (2017). Big data modeling to predict platelet usage and minimize wastage in a tertiary care system. PNAS, 114(43), 11368–11373.

[5] Motamedi M. et al. (2022). Reduction of platelet outdating and shortage by forecasting demand with statistical learning and deep neural networks. JMIR Medical Informatics, 10(2), e29978.

[6] van Dijk N., Haijema R., van der Wal J., Smit Sibinga C. (2009). Blood platelet production: a novel approach for practical optimization. Transfusion, 49.

[7] Platelet inventory management in hospital networks: A reinforcement learning approach (2025). Transportation Research Part E: Logistics and Transportation Review.

[8] Abouee-Mehrizi H., Mirjalili M., Sarhangian V. Platelet inventory management with approximate dynamic programming. INFORMS Journal on Computing. arXiv:2307.09395

[9] The rationale for discarding blood and its components in a tertiary care hospital blood bank in North Karnataka. PMC9720350.

[10] Blood component utilization and discard patterns in Telangana, India: a three-year retrospective study. PMC12822613.

[11] Urs et al. (2026). Timely geographic access to blood banking facilities: a pan-India modelling study. Transfusion Medicine. doi:10.1111/tme.70075

[12] Sankalp India Foundation. Drugs and Cosmetics Rules amended: inter blood bank transfers permitted (amendment of 3 April 2017).

[13] A conformal approach to feature-based newsvendor under model misspecification. arXiv:2412.13159

[14] Open Government Data Platform India. State/UT-wise number of blood units collected (source e-RaktKosh), 2018–2022. data.gov.in

[15] National AIDS Control Organisation. Standards for Blood Banks and Blood Transfusion Services. naco.gov.in

[16] HL7 FHIR BiologicallyDerivedProduct resource (build.fhir.org) and ICCBBA ISBT 128 standard.

[17] Shankaralingappa A. et al. (2025). A 5-year retrospective analysis of discard of blood and its components at a tertiary trauma centre. Asian Journal of Transfusion Science, 19(2).

[18] AI and blood banking: predicting transfusion demand, a systematic review of forecasting approaches. Transfusion Medicine and Hemotherapy. PMC13427359.

[19] Kök A.G., Shang K.H. (2007). Inspection and replenishment policies for systems with inventory record inaccuracy. Manufacturing & Service Operations Management, 9(2).

Verify every reference against the original before citing it in the report or paper; use the primary source (the Rajya Sabha answer, the NACO standards, the journal article) rather than news summaries. Shelf-life values must be confirmed against the current national standards.